PROJECT BUILD SPECIFICATION

Self-Correcting Manufacturing Process Agent

A research-oriented, open-source prototype for AI-assisted CNC process planning, constraint validation, iterative repair and evaluation

Purpose of this document
This brief is intended to be given to Claude (or another coding assistant) as the master specification for designing and implementing the project. It defines the problem, scope, architecture, implementation sequence, testing strategy, evaluation plan, repository structure and expected deliverables.

Project level: Personal research prototype / portfolio project
Primary domain: Manufacturing engineering and CNC process planning
Primary technical themes: Agentic AI, formal constraints, optimization, explainability and reproducible evaluation
Recommended implementation: Python-first MVP, with optional CAD integration in a later phase

Important scope boundary

The first version is a decision-support research prototype. It must not claim that a generated plan is safe, production-ready or suitable for execution on a real machine. Real process planning requires validated geometry, machine/tool data, setup knowledge, cutting parameters, tolerances and expert review. The MVP uses explicit, simplified assumptions and labels them clearly.

1. Project overview

Working title: Self-Correcting Manufacturing Process Agent (SCMPA)

One-sentence description: Build an AI-assisted system that proposes a sequence of manufacturing operations for a simplified component, checks the proposal against explicit manufacturing constraints, diagnoses violations, repairs the plan and validates the revised plan until it is feasible or requires human intervention.

Core research question: Can a constraint-guided agentic workflow improve the feasibility and completeness of generated manufacturing process plans compared with an LLM-only baseline?

The project is not simply an LLM that writes manufacturing steps. Its central contribution is a closed-loop workflow: **generate** → **validate** → **diagnose** → **repair** → **revalidate**. The LLM is responsible for proposing and revising candidate plans. Deterministic Python rules and, later, an optimizer are responsible for checking hard constraints.

1.1 Target user and use case

The intended user is a manufacturing/process engineer who wants an initial draft of an operation sequence and a transparent explanation of whether it satisfies a defined set of planning rules. The prototype should help an engineer inspect and improve a draft; it should not replace engineering approval.

1.2 What the MVP will do

- Accept a structured description of a simple prismatic component and the available manufacturing resources.
- Generate a structured process plan containing operations, tools, machines, feature coverage and precedence relations.
- Validate the plan with deterministic, testable Python rules.
- Return machine-readable violations with evidence and repair hints.
- Ask an AI repair component to propose a revised plan.
- Revalidate the revised plan and stop when it passes, reaches an iteration limit or requires human review.
- Record every candidate, validation result, repair action and final outcome for evaluation.

1.3 What the MVP will not do

- No automatic generation of executable CNC G-code.
- No guarantee of collision-free toolpaths or real-world manufacturability.
- No unverified cutting speeds, feeds, tolerances or machine capabilities.
- No assumption that a generic operation precedence rule is universally correct.
- No autonomous execution or connection to production machinery.
- No medical, safety-critical or production deployment claims.

2. Research framing and hypotheses

Use the following hypotheses as testable starting points. They are not conclusions; the evaluation must determine whether the evidence supports them.

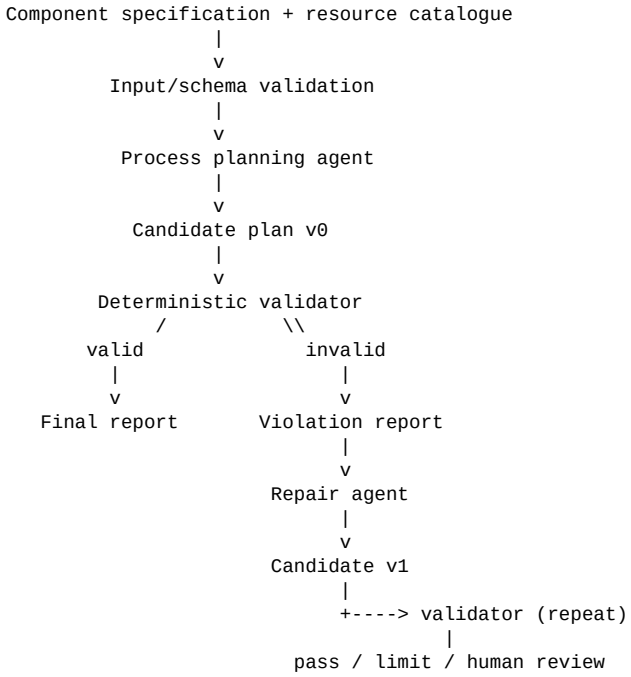
- **H1 — Constraint guidance:** Plans generated with explicit constraint feedback will have a lower hard-constraint violation rate than plans generated by an LLM without validation feedback.
- **H2 — Iterative repair:** A bounded generate–validate–repair loop will convert a measurable proportion of initially invalid plans into valid plans.
- **H3 — Optimization:** Adding a deterministic sequencing optimizer will improve one or more declared plan-quality metrics without violating hard constraints.

- **H4 — Transparency:** Structured violation reports and repair histories will make the system's decisions easier to audit than free-text LLM output alone.

Potential contribution: a reproducible benchmark and evaluation pipeline for constraint-aware process-plan generation and repair, not a claim to have solved general industrial process planning.

3. System scope and workflow

3.1 High-level workflow



3.2 Main software components

Component	Responsibility	Must be deterministic?
Input loader	Load component, feature and resource data; validate schema.	Yes
Planner	Propose candidate operations and dependencies.	No; LLM-assisted
Plan normalizer	Convert model output into canonical typed objects.	Yes
Constraint validator	Check hard rules and produce violation records.	Yes
Repair agent	Use violation evidence to revise the plan.	LLM-assisted
Optimizer	Find a better feasible order or resource assignment.	Yes, for a fixed model
Run recorder	Store prompts, plans, violations, iterations and metrics.	Yes
UI/API	Let a user submit examples and inspect results.	Yes

3.3 Agent responsibilities

- **Planner agent:** proposes a plan from the structured input and the allowed operation/tool/machine catalogue. It must not invent unavailable resources.
- **Repair agent:** receives the candidate plan and exact validator findings. It should make the smallest justified changes and preserve unaffected operations.
- **Optional critic:** checks whether the plan explanation is consistent with the structured plan. It must not override deterministic validator results.
- **Human review gate:** stops autonomous repair when rules are ambiguous, required data is missing, or the iteration budget is exhausted.

3.4 State machine

```
RECEIVED
  -> INPUT_VALIDATED
```

```
-> PLAN_GENERATED
-> PLAN_VALIDATED
     -> ACCEPTED
     -> REPAIR_REQUESTED
          -> PLAN_REPAIRED
          -> PLAN_VALIDATED
               -> ACCEPTED
               -> REPAIR_REQUESTED (if budget remains)
               -> NEEDS_HUMAN_REVIEW (otherwise)
     -> NEEDS_HUMAN_REVIEW (if non-repairable)
```

Use a maximum of 3 repair iterations for the initial MVP. Make this configurable. Prevent infinite loops and detect repeated identical plans.

4. Data model and schemas

Use typed Python models (Pydantic is recommended) and versioned JSON files. Every plan should reference stable IDs for features, operations, tools and machines. Avoid relying on free-text names for validation.

4.1 Component input example

```
{
  "component_id": "demo_bracket_001",
  "description": "Simplified aluminium mounting bracket",
  "material": "aluminium",
  "features": [
    {"feature_id": "F1", "type": "planar_face", "required": true},
    {"feature_id": "F2", "type": "through_hole", "diameter_mm": 8, "required": true},
    {"feature_id": "F3", "type": "pocket", "depth_mm": 5, "required": true}
  ],
  "resources": {
    "machines": [
      {"machine_id": "M1", "capabilities": ["face_milling", "drilling", "pocket_milling"]}
    ],
    "tools": [
      {"tool_id": "T1", "type": "face_mill"},
      {"tool_id": "T2", "type": "drill"},
      {"tool_id": "T3", "type": "end_mill"}
    ]
  },
  "assumptions": [
    "Single setup is assumed for the demonstration.",
    "Tool access and collision checks are not modelled in MVP."
  ]
}
```

This is a toy structured input, not a full CAD-derived engineering definition. Make all assumptions visible in the UI and README.

4.2 Process plan schema example

```
{
  "plan_id": "plan_demo_001",
  "component_id": "demo_bracket_001",
  "operations": [
    {
      "operation_id": "OP1",
      "type": "face_milling",
      "target_features": ["F1"],
      "machine_id": "M1",
      "tool_id": "T1",
      "duration_min": 4.0
    },
    {
      "operation_id": "OP2",
      "type": "drilling",
      "target_features": ["F2"],
      "machine_id": "M1",
      "tool_id": "T2",
      "duration_min": 2.0
    }
  ],
  "precedence": [["OP1", "OP2"]],
  "explanation": "Face milling is scheduled before drilling under the demo rule set."
}
```

4.3 Validator result schema example

```
{
  "valid": false,
  "hard_constraint_violations": [
    {
      "rule_id": "R_TOOL_COMPATIBILITY",
      "operation_id": "OP2",
      "message": "Selected tool type is incompatible with drilling.",
      "evidence": {
        "selected_tool": "T3",
        "required_tool_type": "drill"
      },
      "repair_hint": "Select a compatible drill tool."
    }
  ],
```

```
  "warnings": [],
  "coverage": {
    "required_features": 3,
    "covered_features": 2,
    "missing_features": ["F3"]
  }
}
```

4.4 Required schema principles

- Validate all model output against a schema before using it.
- Reject unknown operation, tool and machine IDs rather than silently accepting them.
- Separate hard errors from warnings and preferences.
- Keep the structured plan as the source of truth; explanations must not change the data.
- Record schema version and dataset/source provenance.
- Use units explicitly (for example, mm and minutes).

5. Manufacturing constraints: design carefully

The validator is the engineering core of the project. Start with a small rule set whose assumptions can be explained and tested. Do not encode domain claims as universal facts when they depend on setup, geometry, tooling or process context.

5.1 Recommended MVP constraints

Rule	Check	Severity
R1 — Valid operation type	Every operation type belongs to the allowed operation catalogue.	Hard
R2 — Valid machine	Every operation references an available machine.	Hard
R3 — Machine capability	Machine capabilities include the requested operation.	Hard
R4 — Tool exists	Every operation references a tool in the available tool catalogue.	Hard
R5 — Tool compatibility	Tool type is compatible with operation type using an explicit compatibility map.	Hard
R6 — Feature coverage	Every required feature is addressed by at least one suitable operation.	Hard
R7 — Valid feature references	Each operation target references a feature in the component input.	Hard
R8 — Precedence validity	Precedence edges reference existing operations and contain no cycle.	Hard
R9 — Precedence order	The sequence respects all declared predecessor relationships.	Hard
R10 — Duplicate handling	Flag duplicate operations on the same feature unless explicitly allowed.	Warning or hard, configurable
R11 — Duration sanity	Durations are positive and within configurable demo bounds.	Hard or warning
R12 — Completeness	All required process-plan fields are present.	Hard

5.2 Important caution about operation order

Do not hard-code a universal rule such as “face milling must always precede drilling.” The valid sequence depends on datums, setups, geometry, workholding, tolerance requirements and the actual process plan. In the MVP, represent precedence as explicit input data or as a clearly labelled pedagogical rule for a specific demo component. Explain the assumptions in the dataset.

5.3 Rule representation

```
Rule(
    rule_id="R_MACHINE_CAPABILITY",
    severity="hard",
    description="Machine must support the operation type",
    check=lambda operation, context:
        operation.type in context.machine(operation.machine_id).capabilities
)
```

In implementation, prefer named validator functions over arbitrary lambda expressions in configuration. Each rule should return structured violations, not just True/False.

5.4 Hard constraints vs soft objectives

- **Hard constraints:** must be satisfied for a plan to be considered valid under the MVP model.
- **Soft objectives:** can be optimized after feasibility, such as fewer setups, fewer tool changes, shorter estimated duration or lower preference penalties.

- Never allow an optimizer to trade away a hard constraint for a shorter plan.
- If weights are used for soft objectives, document their meaning and run sensitivity checks.

6. Planning, repair and optimization logic

6.1 Planner prompt requirements

The planner prompt should instruct the model to:

- Use only the operations, machines and tools supplied in the input catalogue.
- Cover every required feature.
- Return JSON matching the exact schema and no surrounding prose.
- Include explicit precedence relations only when supported by the supplied rules or data.
- List assumptions and uncertainties separately.
- Never claim physical feasibility if geometry, access or collision checks are unavailable.

6.2 Repair prompt requirements

- Treat validator output as authoritative for the defined rule set.
- Fix every reported hard violation if a supported repair exists.
- Make the smallest change possible; preserve valid operations.
- Do not invent new machines, tools, features or manufacturing rules.
- If a violation cannot be resolved from available data, return NEEDS_HUMAN_REVIEW with the missing information.
- Return a complete revised plan in the required schema.

6.3 Repair algorithm

```
def solve(component, max_repairs=3):
    validate_input(component)

    plan = planner.generate(component)
    plan = normalize_and_validate_schema(plan)

    history = []
    seen_plan_hashes = set()

    for iteration in range(max_repairs + 1):
        result = validator.validate(component, plan)
        history.append({"plan": plan, "validation": result})

        if result.valid:
            return accepted_result(plan, history)

        if iteration == max_repairs:
            break

        plan_hash = stable_hash(plan)
        if plan_hash in seen_plan_hashes:
            break
        seen_plan_hashes.add(plan_hash)

        repaired = repair_agent.repair(
            component=component,
            plan=plan,
            violations=result.violations
        )
        plan = normalize_and_validate_schema(repaired)

    return human_review_result(history)
```

6.4 Where OR-Tools fits

Do not use OR-Tools as a decorative dependency. Introduce it only when there is a concrete optimization problem. A suitable next step is a precedence-constrained operation sequencing problem with a simple objective such as

minimizing total estimated duration or setup changes, subject to declared precedence and resource constraints.

- Represent each operation as a task.
- Represent precedence as directed edges.
- Represent eligible machines/tools as allowed assignments.
- Define a transparent objective function.
- Return the optimized sequence and objective value.
- Run the independent validator again after optimization.

6.5 Optimization is not the same as repair

Repair restores feasibility after a detected violation. Optimization improves a feasible plan according to explicit preferences. Keep these as separate modules and report their effects separately in experiments.

7. Data strategy and provenance

7.1 Primary candidate: FabriCAD

Investigate the FabriCAD project and its accompanying documentation as the first candidate source for CAD models and process-planning examples. Before writing an importer, inspect the actual repository structure, data fields, examples and license. Do not assume that every sample contains all information required by this project's schema.

FabriCAD GitHub repository

7.2 Fallback: a transparent synthetic benchmark

If FabriCAD is difficult to parse or does not contain enough labelled examples, create a small synthetic benchmark. Synthetic examples are acceptable for validating software logic, but clearly label them as synthetic and do not present results as industrial performance.

- Create 10–30 simplified component specifications.
- For each component, define required features, available operations, machine/tool catalogue and explicit precedence assumptions.
- Create one or more expert-authored reference plans per component.
- Create corrupted versions by swapping precedence, removing required operations, assigning incompatible tools or referencing unavailable machines.
- Keep generation scripts and random seeds so the benchmark can be reproduced.

7.3 Dataset split and leakage

- Split by component ID, not by individual plan row, so variants of one component do not leak across train/test.
- Keep a fixed test set that is not used to tune prompts or rules.
- Record whether each example is real, derived, or synthetic.
- Do not publish data files whose license prohibits redistribution; publish an importer and instructions instead.

7.4 Data card

Add a DATA_CARD.md documenting source, URL, license, fields used, preprocessing, missing values, synthetic-data generation, intended use and limitations.

8. Evaluation plan

A strong portfolio project must show measurable evidence, not only a polished demo. Build a repeatable experiment script that runs the same examples through all baselines.

8.1 Baselines

System	Description
B0 — LLM-only	Generate one plan; no validator feedback or repair.
B1 — LLM + validator	Generate and validate; report errors but do not repair.
B2 — LLM + repair loop	Generate, validate, repair and revalidate.
B3 — Repair + optimizer (optional)	Use the repair loop, then optimize feasible plans and validate again.

8.2 Metrics

Metric	Definition / reporting guidance
Hard violation rate	Number of plans with at least one hard violation divided by all evaluated plans.
Violation count	Total hard violations per plan; also report by rule ID.
Repair success rate	Initially invalid plans that become valid within the repair budget / initially invalid plans.
Feature coverage	Required features covered by valid operations / total required features.
Precedence accuracy	Agreement with reference precedence relations where references exist.
Sequence similarity	Use a clearly defined metric; do not treat one expert sequence as the only possible valid sequence.
Repair iterations	Mean/median number of repair rounds among repaired cases.
Latency and cost	Wall-clock time and model/API cost, if applicable.
Human review rate	Fraction of cases that correctly stop and request human input.

8.3 Experimental protocol

- Freeze the dataset, rules, model identifier and prompt version for each reported run.
- Run every baseline on the same test components.
- Use multiple random seeds or repeated model runs when generation is stochastic.
- Report mean and spread (or confidence intervals where appropriate), not only the best run.
- Inspect false passes: plans the validator marks valid that are actually wrong under the declared assumptions.
- Inspect false failures: valid alternatives rejected because the rule set is too restrictive.
- Include representative before/after examples with the exact violation and repair history.

8.4 Success criteria for an MVP

Set numerical targets only after a pilot run. A reasonable engineering acceptance criterion is that the validator catches all intentionally injected violations covered by unit tests, that no invalid schema reaches the validator, and that the repair loop terminates safely. For research claims, report actual measured results rather than selecting a target and implying it was achieved.

9. Recommended technology stack

Layer	Recommended choice	Reason
Language	Python 3.11 or newer	Broad ecosystem; typed models and testing.
Schemas	Pydantic	Strict input/output validation.
Agent workflow	LangGraph (optional in MVP)	Explicit stateful generate–validate–repair graph.
LLM	Provider-neutral adapter; local Ollama or API	Avoid locking the project to one provider.
Validation	Pure Python modules	Deterministic, explainable and testable.
Optimization	Google OR-Tools (phase 2)	Constraint programming and scheduling.
CAD	FreeCAD (phase 3)	Optional geometry inspection and feature extraction.
UI	Streamlit	Fast local demonstration.
Testing	pytest	Unit and integration tests.
Experiment tracking	CSV/JSON first; MLflow optional	Reproducible evaluation without early complexity.

Use a provider-neutral interface such as PlannerLLM.generate_plan(...) and RepairLLM.repair_plan(...). The rest of the codebase should not depend on a specific model SDK.

9.1 Repository structure

```
self-correcting-manufacturing-process-agent/
■■■ README.md
■■■ LICENSE
■■■ pyproject.toml
■■■ .env.example
■■■ .gitignore
■■■ docs/
■   ■■■ architecture.md
■   ■■■ research_questions.md
■   ■■■ assumptions_and_limits.md
■   ■■■ DATA_CARD.md
■■■ data/
■   ■■■ examples/
■   ■■■ synthetic/
■   ■■■ README.md
■■■ src/
■   ■■■ scmpa/
■       ■■■ schemas/
■       ■   ■■■ component.py
■       ■   ■■■ resources.py
■       ■   ■■■ process_plan.py
■       ■■■ planning/
■       ■   ■■■ planner.py
■       ■   ■■■ prompts.py
■       ■■■ validation/
■       ■   ■■■ validator.py
■       ■   ■■■ rules/
■       ■■■ repair/
■       ■   ■■■ repair_agent.py
■       ■■■ optimization/
■       ■   ■■■ sequence_optimizer.py
■       ■■■ workflow/
■       ■   ■■■ orchestration.py
■       ■■■ evaluation/
■       ■   ■■■ metrics.py
■       ■   ■■■ run_experiments.py
■       ■■■ app/
■           ■■■ streamlit_app.py
■■■ tests/
■   ■■■ unit/
■   ■■■ integration/
■   ■■■ fixtures/
■■■ scripts/
```

```
■   ■■■ make_synthetic_data.py
■   ■■■ run_demo.py
■■■ reports/
    ■■■ .gitkeep
```

9.2 Engineering quality requirements

- Use type hints and docstrings for public functions.
- Keep prompts in version-controlled files, not hidden inside UI code.
- Use environment variables for API keys; never commit secrets.
- Provide .env.example with placeholder names only.
- Pin or constrain dependencies and document setup commands.
- Use structured logging and stable run IDs.
- Make the core validator runnable without an LLM or internet connection.
- Provide deterministic unit tests for every hard constraint.

10. Step-by-step implementation instructions

Build in this order. Do not start with a complex multi-agent framework or CAD integration.

Milestone 0 — Repository and specification

- Create the repository and Python package.
- Write README with the research question, scope, limitations and planned milestones.
- Create two or three example component JSON files.
- Define Pydantic schemas for component, machine, tool, operation, process plan and validation result.

Acceptance check: malformed input and unknown IDs are rejected with clear errors.

Milestone 1 — Deterministic validator

- Implement the 12 MVP rules in Section 5.
- Return structured violations with rule ID, operation/feature ID, evidence and repair hint.
- Build unit tests for a valid plan and for each individual violation.
- Add cycle detection for precedence graphs and verify sequence order.

Acceptance check: all rule tests pass; validator works offline and without an LLM.

Milestone 2 — Baseline planner

- Implement a planner interface and one LLM provider adapter.
- Pass only the component and allowed resource catalogue to the model.
- Request schema-conforming JSON.
- Validate and normalize every model response before use.
- Add a deterministic mock planner for tests and offline demos.

Acceptance check: planner returns a valid schema or a controlled, understandable failure.

Milestone 3 — Repair loop

- Implement the bounded loop from Section 6.3.
- Pass exact validator output to the repair agent.
- Track all iterations and detect repeated plans.
- Stop on success, maximum attempts, invalid model output or unresolved ambiguity.
- Add integration tests using a mock repair agent.

Acceptance check: intentionally corrupted plans can be repaired in test cases; non-repairable cases stop safely.

Milestone 4 — Experiment runner

- Create a fixed benchmark and test split.
- Run B0, B1 and B2 on identical test cases.
- Compute metrics and save raw results to CSV/JSON.
- Generate tables and plots from saved results.
- Document model, prompts, seeds, costs and limitations.

Acceptance check: one command reproduces the reported experiment from the documented environment.

Milestone 5 — Demo UI

- Build a Streamlit page to select/load a component example.
- Show input resources and assumptions.
- Display initial plan, violations, each repair, and final plan.
- Show a timeline or table of iterations and evaluation metrics.
- Allow download of the final plan and run log as JSON.

Acceptance check: a reviewer can understand the workflow without reading source code.

Milestone 6 — Optional CAD and optimization extensions

- Investigate FabriCAD importer and document the mapping to internal schemas.
- Add OR-Tools sequencing only after defining a concrete objective and constraints.
- Explore FreeCAD-based feature extraction as a separate module.
- Keep CAD-derived features distinct from manually supplied or synthetic features.
- Re-run the validator after every optimization or CAD-to-plan transformation.

11. Master instructions to give Claude

Copy the following instructions into Claude along with this PDF. Ask it to work milestone by milestone and create actual files in your project workspace.

```
You are my senior Python engineer and research-prototype collaborator.
Use this PDF as the authoritative project specification for the
Self-Correcting Manufacturing Process Agent (SCMPA).

Your task is to help me build a clean, reproducible, research-oriented
GitHub project. Do not merely describe code: create and edit the project
files in the workspace when tools allow.

WORKING RULES
1. First inspect the current workspace and report what exists.
2. Read this specification fully before implementation.
3. Do not build the whole system in one huge step. Work milestone by
   milestone in the order in Section 10.
4. Before coding each milestone, briefly state the files you will create
   and the acceptance criteria you will satisfy.
5. After coding, run tests or commands where possible and report actual
   results. Never claim tests passed if they were not run.
6. Prefer simple, modular, typed Python over unnecessary abstractions.
7. Keep the core validator deterministic and independent of any LLM.
8. Use Pydantic schemas and structured JSON for component inputs,
   process plans and validation results.
9. Never allow an LLM to override or falsify validator results.
10. Treat manufacturing rules as assumptions that must be documented.
    Do not claim a universal operation order unless the dataset or
    explicitly stated assumptions support it.
11. Do not generate executable CNC code or claim production readiness.
12. Do not invent dataset fields, licences, benchmark labels or results.
    Inspect source documentation and record uncertainty.
13. Keep API keys out of source control. Create .env.example only.
14. Include unit tests for every hard constraint and integration tests
    for the repair loop.
15. Preserve provenance: identify real, derived and synthetic data.
16. Make the system work with a mock planner so core tests and demos
    do not require an API key.
17. Use a provider-neutral LLM interface so the model provider can be
    changed without rewriting the workflow.
18. Add LangGraph and OR-Tools only when their role is justified.
19. Do not add CAD parsing until the structured-input MVP is working.
20. If a requirement is ambiguous, ask me one focused question or make
    a clearly documented conservative assumption; do not silently guess.

IMPLEMENTATION ORDER
A. Repository, schemas and sample data.
B. Deterministic manufacturing constraint validator.
C. LLM planner and mock planner.
D. Bounded generate-validate-repair workflow.
E. Benchmark, baselines and metrics.
F. Streamlit demonstration and documentation.
G. Optional OR-Tools optimization, FabriCAD importer and FreeCAD
   feature extraction.

FIRST RESPONSE
Start by inspecting the workspace. Then propose a concise implementation
plan for Milestone 0 and Milestone 1, including exact file names.
Do not jump ahead to the full agent or UI before the validator and its
tests are correct.

DEFINITION OF DONE
The MVP is complete only when:
- example component data is schema-validated;
- the validator checks all documented hard constraints;
- each hard rule has positive and negative unit tests;
- the planner can produce structured plans (a mock is sufficient);
- the repair loop is bounded and records every iteration;
- non-repairable cases stop and request human review;
- B0/B1/B2 can be evaluated on the same fixed test set;
- metrics and raw experiment outputs are saved;
- the demo exposes the plan, violations and repair history;
- README explains setup, usage, architecture, data provenance,
  evaluation, limitations and future work;
- tests and demo commands have been run and their results reported.
```

12. Suggested README outline

- Project title and one-sentence summary.
- Problem statement and research question.
- System architecture diagram.
- Demo GIF or screenshots.
- Quick start and environment setup.
- How to run the validator, agent and Streamlit demo.
- Input/output schema examples.
- Constraint catalogue and assumptions.
- Dataset sources, provenance and licenses.
- Evaluation methodology, baselines and results.
- Known limitations and safety disclaimer.
- Roadmap and contribution guidance.

13. Risks and mitigation

Risk	Mitigation
LLM returns malformed JSON	Schema validation, retry policy, mock planner and failure state.
LLM invents tools or operations	Allow-list IDs and reject unknown references.
Rules encode false manufacturing assumptions	Document scope; use explicit component-specific precedence; expert review.
Repair loop oscillates	Maximum iterations, plan hashing and repeated-state detection.
Synthetic results look stronger than real-world evidence	Label synthetic data; avoid industrial generalization claims.
Dataset license restricts redistribution	Publish scripts/instructions and link to source rather than copying data.
Too much scope / project stalls	Complete validator-first MVP before CAD, multi-agent or UI extensions.
API costs or provider dependency	Mock planner, local model option and provider-neutral interface.

14. Final guidance

> The strongest version of this project is not the one with the most agents or the largest technology stack. It is the one with a clear manufacturing problem, a defensible rule model, a robust repair loop and an honest evaluation against baselines.

Prioritize the following order: **correct data model** → **reliable validator** → **measurable repair loop** → **evaluation** → **attractive demo** → **CAD/optimization extensions.**

A small system that catches and repairs well-defined process-planning errors—and proves this with repeatable experiments—is a stronger research portfolio artifact than a large, unreliable “autonomous factory” demo.

End of project specification